

Conservation Corrector — Integration Spec (for SLEEK / PFPL)

Alika M. Parks — for M. Burtscher

September 2026

Reference code: `conserve.h` (public-domain reference; CPU reference + CUDA mapping in comments). This spec is what you need to wire it into GPU SLEEK / PFPL.

What it does

A lossy codec preserves a pointwise error bound but not integrated invariants (mass, energy, momentum). The corrector restores chosen invariants **exactly**, in closed form, from a few stored scalars, touching only the decompressed array — so it is codec-agnostic and drops in after decode. Verified on a reference field: mass and energy drift $7.5\text{e-}5 \rightarrow 3\text{e-}8$ (float32 floor), for two stored scalars.

Three integration points

1. **Before compression — record the targets.** On the original field, compute the invariants you want preserved and store them as side-info:
 - mass + energy: $M0 = \sum x$, $S0 = \sum x^2$ (two doubles per field/block).
 - a linear invariant (e.g. a momentum component): $L0 = \sum x$ (one double).
These are the only bytes the layer adds.
2. **Compress at a slightly tightened error bound.** The corrector's rescale can move a value by up to a small δ , so to keep the *total* error inside the user's requested bound b , run the codec at a tightened bound $b' = b - \delta$ (equivalently $b' = (1-\epsilon) \cdot b$). This is the same bound-adjustment you already do; it works for **any** user-provided b . With SLEEK's power-of-two rounding, if b' lands in the same bucket as b it costs nothing (SLEEK uses the same internal bound); if it crosses to the next bucket it uses that tighter bound, as intended. A conservative ϵ (a few percent) is plenty; the test above showed the rescale staying well under the bound at ϵ in that range. Report the small ratio cost of the tightening.
3. **After decompression — correct in place.** Call the matching routine:
 - `conserve_mass_energy(x, N, M0, S0)` — the workhorse.
 - `conserve_linear(x, N, L0)` — a single linear invariant.

The method (why it is exact and always real)

For mass + energy, the correction is a **decoupled affine**: an offset sets the mean (fixing mass) and a scale about the mean sets the variance (fixing energy). Scaling about the mean leaves the sum unchanged and the offset leaves the variance unchanged, so the two corrections never interfere — the solve is exact and always real (no root-selection, unlike a raw $a \cdot x + b$ match). Full derivation in `conserve.h`.

Taxonomy (for the multi-invariant cases in the paper): a linear invariant $\rightarrow$ an offset; a quadratic invariant $\rightarrow$ a scale-about-mean; invariants on separate fields are independent; k invariants on one field $\rightarrow$ a small k -parameter solve (the 2×2 energy+momentum case is written out in `conserve.h`).

GPU mapping (per field/block)

1. Two reductions: $s = \sum x$, $s2 = \sum x^2$ (fused custom kernel, or `cub::DeviceReduce`; double or Kahan-compensated).
2. Host: `mean`, `var`, `mean0`, `var0`, `a` — five scalars.
3. One elementwise (fused multiply-add) kernel: $x[i] = \text{mean0} + a \cdot (x[i] - \text{mean})$.

Cost is $O(N)$ traffic, one reduce pass + one map pass — negligible next to the codec. It fuses naturally with SLEEK's own decode pass if you want to avoid an extra sweep.

What we need from the integration (for the paper)

- Real coded **ratios** (SLEEK/PFPL full pipeline), layer on/off.
- **Throughput** with and without the corrector, on your GPU system(s) — the E5 numbers.
- Invariant drift, layer on/off, confirming the $\sim$ machine-precision restore holds in the GPU path as it does on CPU.

Verification checklist

- ☐ After correction, $|\sum x - M0|/|M0|$ and $|\sum x^2 - S0|/|S0|$ at the float floor ($\sim 1e-7$ – $1e-8$ in fp32).
- ☐ Max pointwise error stays within the user's bound `b` (confirms `b'` tightening is enough).
- ☐ Throughput overhead is a small fraction of decode time.
- ☐ Corrected-field invariants match the CPU reference (`conserve.h`) bit-for-bit up to fp rounding.